hashlock.



Security Audit

TOPOS Deal (Tokenized Money Infrastructure)

March 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	77
Conclusion	78
Our Methodology	79
Disclaimers	81
About Hashlock	82

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The TOPOS Deal team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

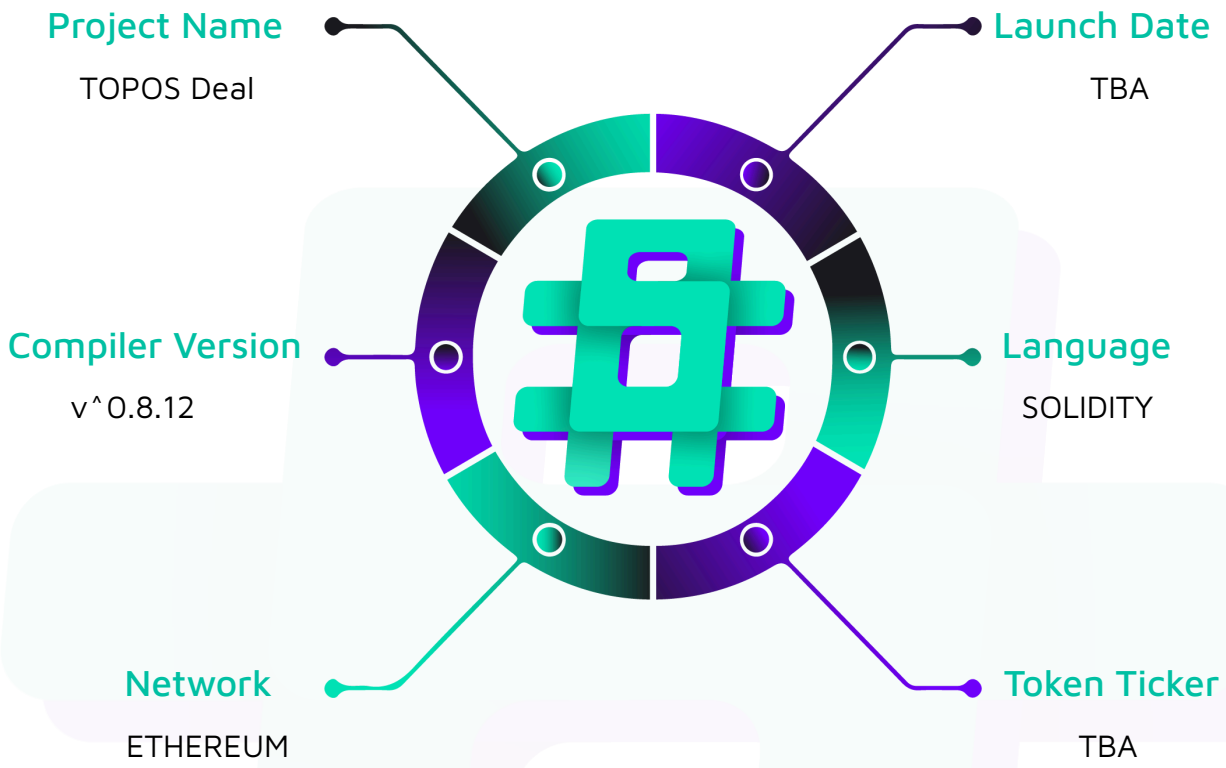
Project Context

Topos is a Web3 financial infrastructure platform focused on building a tokenized payment and clearing operating system that bridges traditional (Web2) and blockchain-based (Web3) financial ecosystems. It aims to enable cross-border payments, remittances, and digital asset transfers using tokenized money, while providing tools such as payment gateways, wallets, and on-chain settlement layers. By integrating blockchain technology with real-world payment networks, Topos positions itself as a global financial infrastructure layer designed to improve efficiency, reduce costs, and expand access to digital financial services.

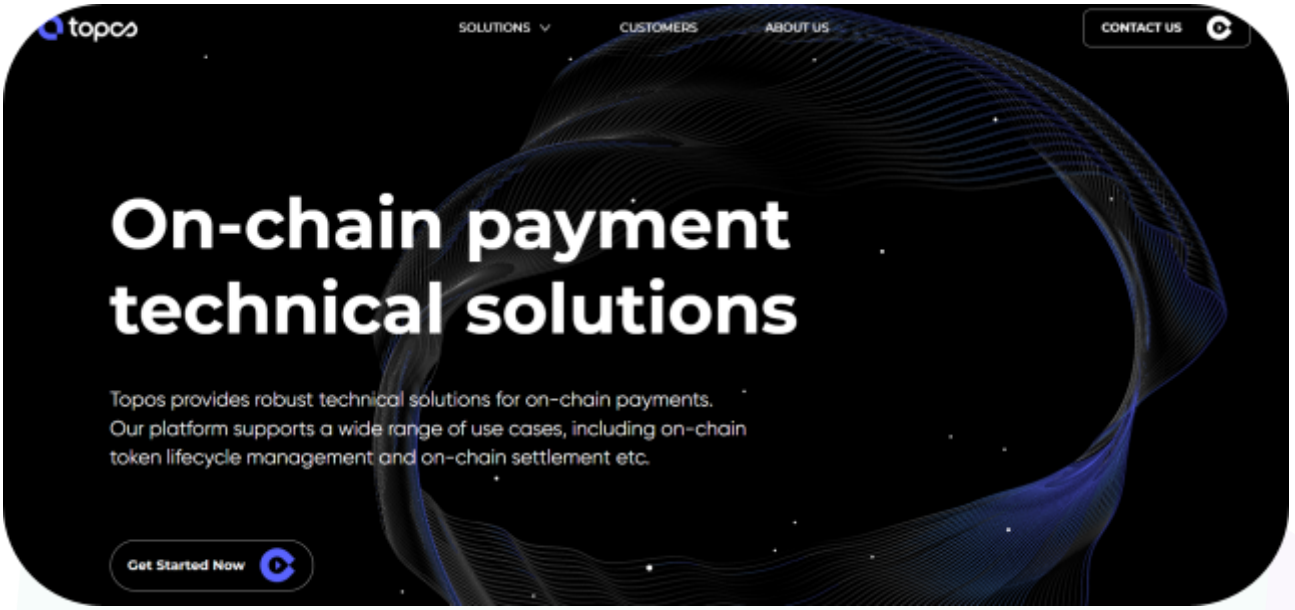
Project Name: TOPOS Deal
Project Type: Tokenized Money Infrastructure
Compiler Version: ^0.8.12
Website: https://topos.one/
Logo:

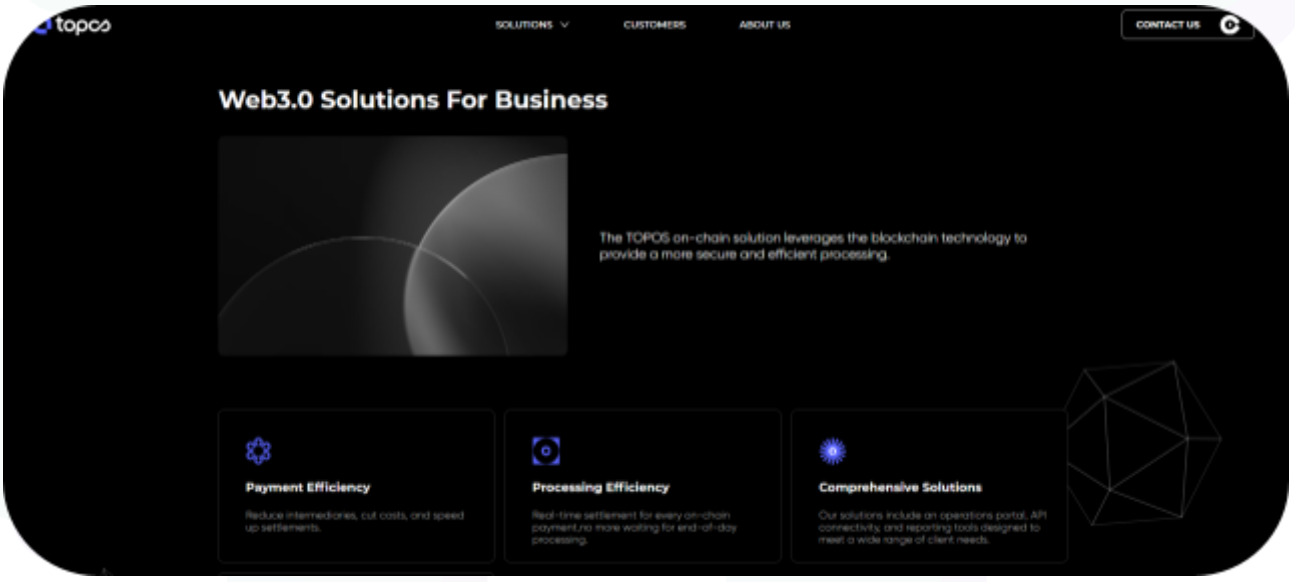


Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the TOPOS Deal project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**TOPOS Deal Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**March, 2026**
Scope 1	remittance-contract
Scope 2	remittance-dex-contract
Scope 3	stablecoin-usd-tokens
Scope 4	topos-wallet-contracts
Audited GitHub Commit Hash	4ec302d891f71c43ef15823568f3ddfb6d0627cf
Fix Review GitHub Commit Hash	ddf0dd16067abe010d3c3a4ac39fb8a255047178

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

7 High severity vulnerabilities

11 Medium severity vulnerabilities

22 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
remittance-contract - Allows the owner to: - Transfer ownership - Set, add, and remove manager addresses - Toggle the business switch - Set remittance fee rate and pay-in source fee rate - Set earning address - Initialize core config - Perform emergency withdrawal - Allows the manager to: - Update exchange rates - Batch update exchange rates - Update support-channel configuration - Allows providers to: - Settle orders - Mark failed settlement - Allows users to: - Create remittance orders	**Contract achieves this functionality.**
remittance-dex-contract - Allows the owner to: - Transfer ownership - Set, add, and remove manager addresses - Toggle the global switch - Reset factory pair state - Update setter roles - Set ransom authorization entries - Perform emergency withdrawal	**Contract achieves this functionality.**

- Allows the manager to: - Set default swap fees - Set per-pair swap fees - Set swap thresholds - Set swap whitelist entries - Update oracle rates - Reset router factory/WETH addresses - Allows users to: - Create prediction/liquidity-style pools - Interact with pair/router entrypoints	
stablecoin-usd-tokens - Allows the owner to: - Transfer ownership - Set masterMinter - Set masterBurner - Set pauser - Set blacklister - Set rescuer - Set router fee parameter - Allows the master minter to: - Configure minters and mint allowances - Remove minters - Allows minters to: - Mint within assigned allowance - Allows the master burner to: - Add burners - Remove burners - Allows burners to: - Burn own balance - Burn from approved balances - Allows the pauser to: - Pause protocol token operations	**Contract achieves this functionality.**

- Unpause protocol token operations - Allows the blacklister to: - Blacklist addresses - Unblacklist addresses - Retire blacklisted funds	
topos-wallet-contracts - Allows wallet owners: - Add, remove, or swap owners - Change signing threshold - Enable/disable modules - Set fallback handler - Lock/unlock wallet - Upgrade wallet implementation - Withdraw wallet deposit from entrypoint - Allows authorized modules to: - Execute wallet calls through module execution paths - Batch execute wallet calls through module execution paths - Allows paymaster owner to: - Set min deposit threshold - Add/remove supported token + oracle mapping - Withdraw ERC20 paymaster funds - Unlock stake - Withdraw stake - Withdraw deposit - Allows entrypoint/stake manager owner to: - Change stake policy params - Allows users to: - Create wallet proxies	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the TOPOS Deal project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the TOPOS Deal project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] BaseWallet#_validateSessionSignature - Unrestricted session keys can execute wallet admin actions through `EntryPoint` module self-calls

Description

Session key validation only checks that the session key exists, is not expired, and has produced a valid signature. It does not restrict what operation the session key is allowed to approve. Since `EntryPoint` is enabled as a wallet module during setup, a session-signed `UserOperation` can be routed through `executeFromModule()`, which then performs a self-call and allows sensitive `onlySelf` wallet functions to execute.

Vulnerability Details

In `BaseWallet._validateSessionSignature()`, the wallet only verifies `session.key`, expiry, and signature validity. There is no check on the target function, selector, value, or action type being approved by the session key.

```
function _validateSessionSignature(SignatureData memory signatureData, bytes32
signMessageHash)
    internal
    view
    returns (bool success)
{
    if (session.key == address(0) || session.expires < block.timestamp) {
        return false;
    }

    return SignatureChecker.isValidSignatureNow(session.key, signMessageHash,
signatureData.signature);
}
```

During wallet setup, the configured `EntryPoint` is enabled as a module. Later, when a session-signed `UserOperation` is executed, `EntryPoint` can call `executeFromModule()`. That function internally performs `address(this).call(data)`, so the next internal wallet call sees `msg.sender == address(this)`, which makes `onlySelf` checks pass.

```
function executeFromModule(bytes memory _data) public onlyAuthorizedModule returns
(bool success) {
    success = _call(_data);
}

function _call(bytes memory data) internal returns (bool success) {
    (success, ) = address(this).call(data);
}
```

Because of this flow, a compromised session key can trigger privileged functions such as `addOwnerWithThreshold()`, `enableModule()`, `upgradeTo()`, `unlock()`, and `withdrawDepositTo()` without collecting the normal owner-threshold signatures.

Impact

A compromised session key can be used to escalate from temporary delegated access to full wallet control. This can let an attacker install a backdoor module, add a malicious owner, upgrade the implementation, unlock the wallet, or withdraw deposits, leading to permanent takeover and loss of funds.

Recommendation

Session keys should be restricted to a limited permission scope and must not be allowed to authorize arbitrary wallet calldata or privileged administrative actions.

Status

Resolved

[H-02] StakeManager#initialize - EntryPoint can be reinitialized through inherited StakeManager.initialize()

Description

The deployed EntryPoint proxy exposes an inherited `initialize(address,uint256,uint256)` function from `StakeManager` that is public and not protected by any initializer or access control. Because the proxy forwards external calls with `delegatecall`, any non-admin user can call this function again and overwrite the proxy's owner and stake policy values.

Vulnerability Details

`EntryPoint.initialize(string,...)` is protected by OpenZeppelin's `initializer`, but it internally calls a different inherited function from `StakeManager` that has no such protection. That inherited function remains publicly callable after deployment.

```
function initialize(address _owner, uint256 _unstakeDelaySec, uint256 _minStakeValue)
public {
unstakeDelaySecThreshold = _unstakeDelaySec;
if (unstakeDelaySecThreshold < MIN_UNSTAKE_DELAY) {
unstakeDelaySecThreshold = MIN_UNSTAKE_DELAY;
}
paymasterStakeThreshold = _minStakeValue;
_MIN_STAKE_VALUE = _minStakeValue;

if (_owner == address(0)) {
    _transferOwnership(_msgSender());
} else {
    _transferOwnership(_owner);
}

}
```

Since the live `EntryPoint` is used through `LogicProxy`, calls from normal users are forwarded to the implementation through fallback using `delegatecall`. This means a later external call to the 3-argument `initialize()` does not affect only the implementation contract, but rewrites the proxy storage itself, including ownership and stake thresholds.

Impact

An attacker can become the owner of the live EntryPoint proxy and gain control over stake-policy management by changing parameters such as `unstakeDelaySecThreshold` and `paymasterStakeThreshold`. This does not directly steal all user funds, but it gives unauthorized control over important protocol configuration and can weaken or disrupt paymaster staking requirements.

Recommendation

Make the inherited `StakeManager.initialize()` inaccessible after deployment by adding proper initialization protection or restricting it to internal use only. Ensure that only the intended top-level initializer can be called on the proxy.

Status

Resolved

[H-03] BaseWallet#isValidSignature - ERC-1271 signature validation ignores the wallet threshold

Description

The wallet's `ERC-1271` implementation accepts a signature from any single owner and does not enforce the configured multisig threshold. As a result, external protocols that rely on `` `isValidSignature()` `` may treat one owner's approval as valid wallet consent even when the wallet is configured to require multiple owners.

Vulnerability Details

The public `ERC-1271` function only checks whether the recovered signer is one of the wallet owners. It does not check how many signatures are required by the wallet threshold.

```
function isValidSignature(bytes32 _hash, bytes memory _signature) external view
returns (bytes4 magicValue) {
    (address signer, ECDSA.RecoverError error) = ECDSA.tryRecover(_hash, _signature);
    if (error == ECDSA.RecoverError.NoError && (isOwner(signer)))
    { return IERC1271.isValidSignature.selector; }
    return SIGNATURE_INVALID;
    }
```

This is different from the wallet's normal owner-signature validation used for `UserOperations`, where the contract validates signatures against the configured `threshold`.

```
function _validateOwnerSignature(SignatureData memory signatureData, bytes32
signMessageHash, bytes32 userOpHash) internal view returns (bool success) {
    if (threshold == 0) { return false; }
    bytes memory signedData = userOpHash.packSignedData(signatureData);
    return _validateNSignatures(signMessageHash, signedData, signatureData.signature,
threshold);
    }
```

Because of this mismatch, a wallet configured as `` `2-of-3` `` can still return a valid `ERC-1271` result for a message signed by only one owner.

Impact

A single owner can authorize offchain actions that depend on `ERC-1271`, such as permits, listings, orders, or other signed intents, even when the wallet is supposed to require multiple owner approvals. This breaks the expected multisig security model for integrations that trust `` `isValidSignature()` `` as the wallet's approval check.

Recommendation

Make the `ERC-1271` validation logic consistent with the wallet's multisig threshold so that contract signature verification enforces the same approval policy as normal wallet operations.

Status

Resolved

[H-04] BasePaymaster#initialize - Proxy paymasters can be re-initialized, allowing ownership takeover of the live proxy

Description

The paymaster proxy can be initialized again after deployment because `BasePaymaster.initialize(address,address)` is publicly callable and is not protected by any one-time initialization guard. This allows an attacker to call the inherited base initializer on the proxy and overwrite both the stored owner and the stored EntryPoint address.

Vulnerability Details

`SponsorPaymaster` and `TokenPaymaster` use guarded child initializers, but both of them internally call the unguarded base initializer. The problem is that the base initializer remains externally reachable through the proxy even after the normal initialization has already happened. Since the proxy holds the real storage, a second call updates the live paymaster state and gives control to the attacker.

```
function initialize(address _entryPoint, address _owner) public {
    require(address(_entryPoint) != address(0), "BasePaymaster: invalid entrypoint
addr");
    _IEntryPoint = IEntryPoint(_entryPoint);
    if (_owner == address(0)) {
        _transferOwnership(_msgSender());
    } else {
        _transferOwnership(_owner);
    }
}
```

Impact

An attacker can become the owner of the live paymaster proxy and use owner-only functions such as `withdrawTo()`, `unlockStake()`, `withdrawStake()`, and in `TokenPaymaster`, `withdrawToken()`, which can lead to theft of deposited funds, withdrawal of stake after the unlock delay, and full loss of administrative control over the paymaster.

Recommendation

Ensure the initialization logic can only be executed once on the proxy and prevent the base initializer from remaining independently callable after deployment.

Status

Resolved

[H-05] BasePaymaster#addStake - BasePaymaster.addStake() Missing onlyOwner Allows Permanent Stake Lock

Description

The `addStake()` function in `BasePaymaster.sol` lacks access control, allowing anyone to call it and permanently lock the paymaster's staked `ETH` by setting an irreversible maximum

Vulnerability Details

The `addStake()` function at `BasePaymaster.sol` is external payable with no onlyOwner modifier, unlike all other stake management functions (`unlockStake`, `withdrawStake`, `withdrawTo`) which are `onlyowner()` restricted.

```
function addStake(uint32 extraUnstakeDelaySec) external payable {

require(extraUnstakeDelaySec > 0, "BP: invalid extraUnstakeDelaySec");

_IEntryPoint.addStake{value: msg.value}(extraUnstakeDelaySec);

}
```

This calls `StakeManager.addStake()` which enforces a monotonically increasing unstake delay at `StakeManager.sol:115`:

require(`_unstakeDelaySec >= info.unstakeDelaySec`, "SM: cannot decrease unstake time");

Since the delay can never be decreased, an attacker can call `paymaster.addStake{value: 0}(type(uint32).max)` to set the unstake delay to ~`136` years. The attacker doesn't need to send any `ETH` if the paymaster already meets the minimum stake threshold. After this, even the owner calling `unlockStake()` results in a withdrawal time `136` years in the future, making the staked `ETH` effectively unrecoverable.

Impact

An attacker can permanently freeze the paymaster's staked `ETH` with a single permissionless call at zero cost. The paymaster continues to operate normally and deposits remain unaffected, but the staked `ETH` becomes permanently locked and unrecoverable by the owner.

Recommendation

Add the `onlyOwner` modifier to `addStake()`, consistent with all other stake management functions in `BasePaymaster`.

Status

Resolved

[H-06] RemitPair#mint - Liquidity deposits do not mint LP shares and burn does not redeem underlying assets

Description

Router transfers user assets into the pair and then calls `mint(to)`, but `RemitPair.mint` does not mint any LP token shares. Likewise, `removeLiquidity` depends on `RemitPair.burn(to)`, but `burn` does not burn LP tokens.

Vulnerability Details

In the deposit flow (`_RemitRouter.addLiquidity_`), assets are moved to the pair before mint is called:

```
TransferHelper.safeTransferFrom(tokenA, msg.sender, pair, amountA);
TransferHelper.safeTransferFrom(tokenB, msg.sender, pair, amountB);
liquidity = IRemitPair(pair).mint(to);
```

But in `RemitPair.mint`, there is no LP minting path (`_mint` is never called, `liquidity` is never assigned):

```
function mint(address to) external lock returns (uint liquidity) {
    ...
    _update(balance0, balance1, _reserve0, _reserve1);
    if (feeOn) kLast = uint(reserve0).mul(reserve1);
    emit Mint(msg.sender, amount0, amount1);
}
```

In the withdrawal flow, router transfers LP tokens to pair then calls `burn(to)`, but `burn` does not burn LP shares and does not transfer token0/token1 to `to`:

```
function burn(address to) external lock returns (uint amount0, uint amount1) {
    ...
    uint balance0 = IERC20(_token0).balanceOf(address(this));
    uint balance1 = IERC20(_token1).balanceOf(address(this));
    //uint liquidity = balanceOf[address(this)];
    emit Burn(msg.sender, amount0, amount1, to);
}
```

Impact

Liquidity providers can send assets into pools without receiving LP ownership. The normal `removeLiquidity` and `burn` redemption path is broken, so funds can become practically unrecoverable through standard flows. AMM accounting and user's assumptions are not reliable for liquidity operations.

Recommendation

Make `mint` actually mint LP shares by calculating `liquidity`, handling minimum locked liquidity, and calling `_mint`.

Also make `burn` actually redeem by using the LP balance held by the pair, calculating proportional token outputs, calling `_burn`, transferring `token0` and `token1` to the receiver, and then syncing reserves.

Status

Resolved

[H-07] SponsorPaymaster#_validatePaymasterUserOp - SponsorPaymaster has no per-sender authorization, allowing any wallet to exhaust the paymaster's ETH.

Description

`_validatePaymasterUserOp` returns `SIG_VALIDATION_SUCCESS` for every sender unconditionally. There is no whitelist, no pre-deposit requirement, and no per-account credit check. Any wallet that references this paymaster in a `UserOperation` is accepted, and the `needSponsored` flag is hardcoded `true` in the returned context, so the funding path in `_postOp` always executes.

Vulnerability Details

The validation function approves every sender without any authorization gate:

```
function _validatePaymasterUserOp(...) internal view override
    returns (bytes memory context, uint256 validationResult)
{
    if (_IEntryPoint.balanceOf(address(this)) <= _MIN_DEPOSIT_THRESHOLD) {
        require(address(this).balance >= requiredPreFund, ...);
    }
    return (abi.encode(userOp.sender, requiredPreFund, true),
SIG_VALIDATION_SUCCESS);
}
```

The `_MIN_DEPOSIT_THRESHOLD` check is effectively inactive in normal operation because of how `_postOp` replenishes the deposit. After each sponsored op, `_postOp` sends `actualGasCost` `ETH` from the paymaster's own raw balance into its EntryPoint deposit via `depositTo`. The EntryPoint then refunds the unused `prefund - actualGasCost` back to the paymaster's deposit. The net result is that the `EntryPoint` deposit is continuously topped up and never drops below the threshold, while the paymaster's raw `ETH` decreases by `actualGasCost` per operation.

An attacker with any valid `EIP-4337` wallet submits operations using this paymaster, pays nothing, and has their gas cost silently recorded as a debt in `_sponsoredAccountFunds`. Since there is no enforcement preventing new operations

before existing debt is repaid, the attacker repeats until the raw `ETH` balance is exhausted. At that point `_postOp` reverts at the `require(address(this).balance >= _deposit)` check, which causes the EntryPoint to fail the operation and brick the paymaster for all other users.

Impact

The paymaster's entire raw `ETH` balance is drainable by any wallet at no cost to the attacker. The debt tracking in `_sponsoredAccountFunds` records the obligation but enforces nothing. Once the raw `ETH` is gone, the paymaster can no longer replenish its EntryPoint deposit and stops functioning for every legitimate user.

Recommendation

Add a per-sender authorization check in `_validatePaymasterUserOp` before approving sponsorship. The simplest fix consistent with the existing accounting model is to require that the sender has a positive `_compensateFunds` pre-deposit before being eligible for sponsorship, turning the current open model into an opt-in funded one. Alternatively, maintain an owner-managed whitelist of approved senders.

Status

Resolved

Medium

[M-01] LogicProxy#constructor - Uninitialized proxy can be taken over before first initialization

Description

The proxy is deployed with a valid implementation address, but it is not initialized in the same transaction. During this gap, any non-admin account can call the implementation's `initialize()` function through the proxy and become the owner while also setting all privileged roles to attacker-controlled addresses.

Vulnerability Details

The proxy constructor only stores the implementation and admin addresses, and does not execute any initialization call:

```
constructor(address implementationContract)
UpgradeabilityProxy(implementationContract)
{
_setAdmin(msg.sender);
}
```

At the same time, the proxy forwards non-admin calls to the implementation through fallback, while blocking only the admin from doing so:

```
fallback() external payable {
_fallback();
}

function _willFallback() internal override {
require(msg.sender != _admin(), "Cannot call fallback function from the proxy
admin");
}
```

The implementation exposes a public one-time initializer:

```
function initialize(...) public initializer {
masterMinter = newMasterMinter;
```

```
masterBurner = newMasterBurner;
pauser = newPauser;
blacklister = newBlacklister;
rescuer = newRescuer;
setOwner(newOwner);
}
```

Because initialization is not atomic with deployment, the first non-admin caller who reaches `initialize()` through the proxy can permanently set the owner and all sensitive roles before the legitimate operator does. The repository's own tests and scripts also show deployment and initialization happening as separate steps, which confirms the race window is real rather than theoretical.

Impact

An attacker can fully take over the proxy at deployment time by becoming the owner and assigning all privileged roles such as master minter, master burner, pauser, blacklister, and rescuer. This can lead to unauthorized minting, censorship, emergency control abuse, asset recovery abuse, and permanent loss of protocol control.

Recommendation

The proxy should be initialized atomically during deployment so no external account can front-run the first initialization call. Any deployment flow that leaves the proxy live but uninitialized should be treated as unsafe.

Status

Resolved

[M-02] BaseWallet#execute - Failed wallet actions are reported as successful at the module and EntryPoint level

Description

When a downstream target call fails inside `BaseWallet.execute()`, the function does not revert and only emits `ExecutionFailure`. Because the outer self-call still succeeds, `ModuleManager` reports success and the `EntryPoint` execution path can mark the full `UserOperation` as successful even though the intended wallet action failed.

Vulnerability Details

`BaseWallet.execute()` performs the target call through `_execute()`, but if that call fails it only emits an event and does not revert.

`function execute(address target, uint256 value, bytes memory data, Enum.Operation operation) external onlySelf onlyUnlocked { bool success = _execute(target, value, data, operation, type(uint256).max); if (success) { emit ExecutionSuccess(address(this), target, value, uint8(operation)); } else { emit ExecutionFailure(address(this), target, value, uint8(operation)); } }`

`ModuleManager` then checks only whether the self-call to the wallet reverted. Since `BaseWallet.execute()` returns normally even after a failed target call, the self-call returns `true` and `ExecuteFromModuleSuccess` is emitted.

`function _call(bytes memory data) internal returns (bool success) { bytes memory returnData; (success, returnData) = address(this).call(data); if (success) { emit ExecuteFromModuleSuccess(msg.sender); } else { emit ExecuteFromModuleFailed(msg.sender, returnData); } }`

At the `EntryPoint` level, success is determined by whether the wallet call reverted. Since the wallet call itself succeeds, the full `UserOperation` can remain on the success path even though the actual transfer, swap, bridge, or delegatecall failed.

Impact

This can mislead relayers, paymasters, monitoring systems, or integrations that trust the top-level execution result. A failed downstream wallet action may still appear as a successful module execution and successful `UserOperation`, causing incorrect accounting, false success reporting, and unreliable operational monitoring.

Recommendation

Make downstream execution failures propagate upward so that failed wallet actions cannot be reported as successful by the module layer or EntryPoint flow.

Status

Resolved

[M-03] SponsorPaymaster#_removeSponsorAccountFunds - Missing `break` in `_removeSponsorAccountFunds()` causes infinite loop when sponsored funds reach zero

Description

The `while(true)` loop in `_removeSponsorAccountFunds()` lacks a `break` statement after unlinking a sponsored account from the linked list, causing an infinite loop and gas exhaustion whenever `compensate()` fully repays an account's sponsored debt.

Vulnerability Details

When `compensate()` is called with an amount that reduces a sponsored account's funds to exactly zero, the function enters a `while(true)` loop to remove the account from the linked list. After finding the target account and unlinking it at lines 159-161, the code sets `_sponsoredAccounts[_addr] = address(0)` but does not break out of the loop. Execution falls through to `cursor = _addr`, and on the next iteration, `_sponsoredAccounts[cursor]` reads `address(0)` (the default mapping value), which itself maps to `address(0)` again — creating an infinite loop that consumes all gas and reverts.

```
if (_sponsoredAccountFunds[account] == 0) {
    address cursor = _SENTINEL_ACCOUNT;
    while (true) {
        address _addr = _sponsoredAccounts[cursor];
        if (_addr == account) {
            address next = _sponsoredAccounts[_addr];
            _sponsoredAccounts[cursor] = next;
            _sponsoredAccounts[_addr] = address(0);
            // @audit missing break here
        }
        cursor = _addr;
    }
}
```

Impact

The `compensate()` function will always revert when attempting to fully repay a sponsored account's debt (reducing the balance to zero), meaning sponsored accounts can never be properly removed from the linked list. However, the paymaster's core `ERC-4337` operations (`_validatePaymasterUserOp` and `_postOp`) are completely unaffected since they never call `_removeSponsorAccountFunds`. A trivial workaround exists by repaying slightly less than the full balance.

Recommendation

Add a break statement immediately after unlinking the account from the linked list.

```
if (_addr == account) {
    address next = _sponsoredAccounts[_addr];
    _sponsoredAccounts[cursor] = next;
    _sponsoredAccounts[_addr] = address(0);
    break;
}
```

Status

Resolved

[M-04] DexContract#settleToken - USDT orders can be misclassified as AEDB orders when `inAmount > settleAmount`

Description

The contract uses `inAmount == dexUsdtAmount` to decide whether an order is a `USDT` order or an `AEDB` order. For orders created through `swapTokenForUSDT()`, this breaks when `_inAmount > _settleAmount`, causing the settlement logic to use the wrong token branch.

Vulnerability Details

In `swapTokenForUSDT()`, the contract allows `_inAmount >= _settleAmount` and stores them separately. It sets `orders[_orderNo].dexUsdtAmount = _settleAmount` and `orders[_orderNo].inAmount = _inAmount`. Later, `settleToken()` checks `if (orders[_orderNo].inAmount == orders[_orderNo].dexUsdtAmount)` to decide whether the platform fee should be paid in `targetTokenContract` or `sourceTokenContract`. If `_inAmount > _settleAmount`, this condition becomes false even though the order was created as a `USDT` order, so the contract enters the `AEDB` branch and tries to send the platform fee in the wrong token.

Impact

A valid `USDT` order can fail during settlement because the contract attempts to transfer the platform fee in `AEDB` instead of `USDT`. If the contract does not hold enough AEDB, settlement reverts; if it does hold `AEDB` from other orders, unrelated funds can be consumed incorrectly.

Recommendation

Store the input token type explicitly at order creation and use that stored value during settlement instead of inferring order type from `inAmount == dexUsdtAmount`. Also, if `USDT` orders are expected to require equality, enforce `_inAmount == _settleAmount` at creation.

Status

Resolved

[M-05] BaseWallet#_validateNSignatures - Contract-owner signatures in wallet validation always fail

Description

The wallet's owner-signature validation path is broken for contract owners that use the `v == 0` ERC-1271 signature format. Because of an incorrect hash check, the code always reverts before calling the contract owner's `isValidSignature()` function.

Vulnerability Details

In `BaseWallet._validateOwnerSignature()`, the code builds `signedData = userOpHash.packSignedData(signatureData)` and passes both `signMessageHash` and `signedData` into `_validateNSignatures()`. Inside the `v == 0` branch, the code checks `require(keccak256(_data) != _dataHash, "wallet: invalid data hash")`. But here `_dataHash` is already `keccak256(_data)`, so this condition can never be true and the function always reverts before reaching `IERC1271(currentOwner).isValidSignature(...)`.

```
bytes memory signedData = userOpHash.packSignedData(signatureData);
return _validateNSignatures(signMessageHash, signedData, signatureData.signature,
threshold);
```

```
if (v == 0) {
    require(keccak256(_data) != _dataHash, "wallet: invalid data hash");
    require(
        IERC1271(currentOwner).isValidSignature(_dataHash, contractSignature)
            == IERC1271.isValidSignature.selector,
        "wallet: invalid eip-1271 signature"
    );
}
```

Impact

Any wallet that uses a smart contract as an owner cannot successfully validate owner signatures through this path. This breaks compatibility with contract-based owners such as multisigs or other smart wallets and can make legitimate `UserOperations` fail.

Recommendation

Fix the invalid hash check in the `v == 0` branch so that contract-owner signatures can reach the `ERC-1271` validation call and be verified correctly.

Status

Resolved

[M-06] RemitERC20#permit - RemitERC20.permit() accepts signatures for a hardcoded EIP-712 chain ID

Description

`RemitERC20.permit()` validates signatures against two `EIP-712` domain separators, and one of them uses a hardcoded chain ID `210425` instead of the current chain ID. This breaks the expected chain-specific binding of permit signatures.

Vulnerability Details

In `RemitERC20`, the constructor creates both a normal domain separator using `chainid()` and another separator using the fixed value `210425`. The `permit()` function first checks signatures against the hardcoded domain and only falls back to the normal domain if that check fails. Because of this, a signature created for the `210425` domain can still be accepted on a deployment on another chain if the contract address and nonce are the same.

```
DOMAIN_SEPARATOR_NEW = keccak256(
    abi.encode(
        keccak256("EIP712Domain(string name,string version,uint256 chainId,address
verifyingContract)"),
        keccak256(bytes(name)),
        keccak256(bytes("1")),
        210425,
        address(this)
    )
);
```

```
bytes32 digest = keccak256(
    abi.encodePacked(
        "\x19\x01",
        DOMAIN_SEPARATOR_NEW,
        keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, value, nonces[owner],
deadline))
    )
);
```

Impact

A valid permit signature intended for chain `210425` can be replayed on another chain and set the same allowance there if the LP token is deployed at the same address and the nonce matches. This does not by itself show direct theft through the router, but it does allow unintended cross-chain approvals and weakens `EIP-712` permit security.

Recommendation

Remove the hardcoded-chain-ID domain path and only accept signatures bound to the current runtime `chainid()`.

Status

Resolved

[M-07] RemitRouter#getAmountIn - RemitRouter.getAmountIn calls getAmountOut instead of getAmountIn

Description

The router's `getAmountIn` function, which should return the input amount required to receive a given output, instead calls `RemitLibrary.getAmountOut`, which computes output for a given input. The return value is semantically inverted.

Vulnerability Details

The public `getAmountIn` view function delegates to `RemitLibrary.getAmountOut` instead of `RemitLibrary.getAmountIn`:

```
function getAmountIn(uint amountOut, uint reserveIn, uint reserveOut, uint rateIn,
uint rateOut,
    uint decimalIn, uint decimalOut, uint mode, uint value) public pure override
returns (uint amountIn) {
    return RemitLibrary.getAmountOut(amountOut, reserveIn, reserveOut, rateIn,
rateOut, decimalIn, decimalOut, mode, value);
}
```

`RemitLibrary.getAmountOut` computes the output for a given input by multiplying with the exchange rate. `RemitLibrary.getAmountIn` does the inverse, it divides the desired output by the rate to get the required input.

Impact

Any caller relying on `getAmountIn` to compute the required input amount for an exact output swap will receive a wrong value. Off-chain integrators, aggregators, or UIs querying this function to estimate input cost will miscalculate, causing users to underpay (causing the swap to revert with `INSUFFICIENT_INPUT_AMOUNT`) or overpay depending on the rate direction.

Recommendation

Change the delegated call to `RemitLibrary.getAmountIn(amountOut, reserveIn, reserveOut, rateIn, rateOut, decimalIn, decimalOut, mode, value)`.

Status

Resolved

[M-8] RemitPair#setSwapWhite - setSwapWhite and setRansomer only append to whitelist arrays and never remove stale entries

Description

Both whitelist setter functions mutate mapping permissions, but array bookkeeping is append-only.

There is no removal path for `swapWhites` or `ransomWhites`, so once an address is pushed, it stays in the array even if later disabled in the mapping.

Vulnerability Details

```
function setSwapWhite(address _swapper, bool _permission) onlyManager external {
        swapWhiteList[_swapper] = _permission;
        swapWhites.push(_swapper);
        emit UpdateSwapWhite(msg.sender, _swapper);
    }
```

```
function setRansomer(address _ransomer, bool _permission) onlyOwner external {
        ransomWhiteList[_ransomer] = _permission;
        if(ransomWhiteList[_ransomer]){
            ransomWhites.push(_ransomer);
        }
        emit UpdateRansomWhite(msg.sender, _ransomer);
    }
```

`setSwapWhite` appends even when disabling (`_permission = false`).
`setRansomer` appends only when enabling, but still has no removal logic when disabling.

In both cases, the arrays become stale over time and do not represent current authorization state.

Impact

`swapAuthsArr()` and `ransomAuthsArr()` can return misleading lists containing disabled addresses and duplicate entries. Repeated enable/disable operations cause unbounded array growth and make array-based monitoring unreliable.

Recommendation

Add removal logic in the setter flows, or introduce dedicated remove functions that keep arrays consistent with mapping state. If append-only history is intended, document that the arrays are not authoritative and require consumers to check `swapWhiteList`/`ransomWhiteList` directly.

Status

Resolved

[M-9] RemitFactory#resetFactory - RemitFactory.resetFactory clears the full pair enumeration while leaving most pair mappings intact

Description

`delete allPairs` zeroes the entire dynamic array in one operation, removing every pair from the enumeration. But only the single `(tokenA, tokenB)` mapping entry passed by the caller gets cleared. Every other deployed pair remains reachable via `getPair` and its underlying contract keeps operating, but none of them appear in `allPairs` anymore.

Vulnerability Details

```
function resetFactory(address tokenA, address tokenB) external onlyOwner {
    delete allPairs;
    delete getPair[tokenA][tokenB];
    delete getPair[tokenB][tokenA];
}
```

After this call, `allPairsLength()` returns zero while other pairs are still live with real reserves. Any code or off-chain tooling that enumerates pairs through `allPairs` will see an empty protocol. At the same time, attempting to re-create any of the non-cleared pairs via `createPair` will revert with `Remit: PAIR_EXISTS` because those mapping entries were never removed, making recovery impossible without a second manual call per pair.

Impact

The enumeration state and the mapping state are permanently out of sync after a reset. Existing liquidity in non-cleared pairs becomes invisible to any consumer that relies on `allPairs`, and those pairs cannot be re-registered without first clearing their mappings individually.

Recommendation

Either iterate and clear all pair mapping entries before deleting the array, or change the function to only remove the one specified pair and its `allPairs` entry rather than wiping the entire array.

Status

Resolved

[M-10] DexContract#swapToken - No on-chain settlement deadline leaves user funds permanently locked if a provider goes silent

Description

The `_deadline` parameter was renamed to `_platformFee` and its value is now stored in `orders[_orderNo].deadline` to record the platform fee rather than a settlement cutoff. There is no timestamp check anywhere in `settleToken` or `swapToken`. Once a user submits an order, their tokens are held in the contract and the only paths to `SETTLE_FAIL` or `REFUND_SUCCESS` both require the provider to act first.

Vulnerability Details

```
// swapToken parameter, original deadline slot repurposed
uint256 /*_deadline*/ _platformFee

// stored in order struct as fee, not a timestamp
orders[_orderNo].deadline = _platformFee;
```

```
function settleToken(...) external nonReentrant switchOn lock {
    require(orders[_orderNo].status == uint256(Status.INIT), "order is not pending");
    require(msg.sender == orders[_orderNo].provider, "msg.sender not equal
provider");
    // no deadline check
```

`refundSettle` requires status `SETTLE_FAIL`, which only the provider can set via `abnormalTokenSettle`. If a provider stops responding, the order stays in `INIT` indefinitely. The only remaining path to recovery is the contract owner calling `withdraw` directly, which is a centralized manual override.

Impact

User funds are permanently locked at the contract level if a provider abandons an order in `INIT` status.

Recommendation

Store `block.timestamp + maxSettlementWindow` as the actual deadline in the `Order` struct at order creation. Add a `forceRefund` function that any caller can invoke once the deadline passes and the order is still in `INIT`, returning funds to the original sender.

Status

Resolved

[M-11] SponsorPaymaster#_removeSponsorAccountFunds - Sponsor fund cleanup can brick compensation when balance hits zero

Description

In `SponsorPaymaster`, linked-list cleanup inside `_removeSponsorAccountFunds()` uses `while (true)` and never exits after unlinking the target account. When compensation reduces sponsored funds to exactly zero, execution enters that cleanup path and loops until gas is exhausted, so the transaction reverts.

Vulnerability Details

```
if (_sponsoredAccountFunds[account] == 0) {
    address cursor = _SENTINEL_ACCOUNT;
    while (true) {
        address _addr = _sponsoredAccounts[cursor];
        if (_addr == account) {
            address next = _sponsoredAccounts[_addr];
            _sponsoredAccounts[cursor] = next;
            _sponsoredAccounts[_addr] = address(0);
            // missing break/return
        }
        cursor = _addr;
    }
}
```

`compensate(account)` calls `_removeSponsorAccountFunds` in both branches. If `msg.value` is exactly equal to current sponsored funds, or greater than it (the function removes `sfunds` in that branch), balance becomes zero and this loop is hit. Because there is no break condition, the cleanup never terminates and compensation reverts.

Impact

Full repayment of sponsored funds cannot be processed. Over-repayment that should clear sponsored debt and move excess into compensation funds also reverts. In practice, only partial repayment paths succeed, which can block normal debt cleanup workflow.

Recommendation

Replace the infinite loop with bounded traversal and exit immediately after unlinking the target account. Also add an explicit terminal condition if the account is not found and include tests for exact-zero repayment, over-repayment, and partial repayment.

Status

Resolved

Low

[L-01] UpgradeProxy#upgradeToAndCall - upgradeToAndCall does not return the original revert reason

Description

The `upgradeToAndCall()` function performs a low-level self-call and only checks whether it succeeded. If the initialization or setup call fails, the proxy always reverts with the generic message `"call failed"` instead of bubbling up the original revert reason from the implementation. This makes upgrade failures harder to diagnose and can slow down incident response or deployment troubleshooting.

Recommendation

Bubble up the original revert data from the failed low-level call instead of replacing it with a generic error message.

Status

Resolved

[L-02] UpgradeProxy#upgradeToAndCall - upgradeToAndCall uses the proxy as msg.sender during migration calls

Description

The `upgradeToAndCall()` function performs a self-call after upgrading, so the migration or initializer logic runs with `msg.sender` equal to the proxy address instead of the admin. In the current codebase this is not an active exploit because the existing initialization logic does not rely on `msg.sender` for ownership or role assignment, so the code handles this safely today. However, this becomes a future upgrade risk if a new implementation or migration function uses `_msgSender()` or `onlyOwner` assumptions during setup, since authority could be assigned incorrectly or the migration could fail unexpectedly.

Recommendation

Keep upgrade-time initialization and migration logic independent of `msg.sender`, and use explicit parameters for privileged role assignment so future implementations do not rely on caller context during `upgradeToAndCall()`.

Status

Resolved

[L-03] LogicProxy#receive - Plain ETH sent to the proxied EntryPoint is accepted but not credited as deposit

Description

The deployed `EntryPoint` is used through `LogicProxy`, and the proxy defines its own `receive()` function that only emits an event. Because of this, sending `ETH` with empty calldata to the proxy does not reach `StakeManager.receive()`, so `depositTo(msg.sender)` is never executed. As a result, the `ETH` is accepted by the contract but no deposit balance is credited to the sender, which can cause the funds to become unrecoverable through the normal deposit withdrawal flow.

Recommendation

Ensure the proxy preserves the implementation's receive-based deposit behavior, or otherwise reject plain `ETH` transfers so users and integrations cannot mistakenly send funds through an unsupported path.

Status

Resolved

[L-04] AdminUpgradeProxy#_willFallback - AdminUpgradeProxy is not actually transparent because the admin fallback guard is never executed

Description

`AdminUpgradeProxy` overrides `_willFallback()` to prevent the proxy admin from reaching implementation logic through fallback, but `UpgradeProxy._fallback()` never calls `_willFallback()`. Because of this, the intended transparent-proxy restriction is not enforced, and the admin can still reach implementation functions through fallback when calling selectors that are not handled by the proxy itself. This breaks the documented admin and operator separation expected from the proxy design.

Recommendation

Ensure the fallback path executes `_willFallback()` before delegating so the admin restriction is actually enforced at runtime.

Status

Resolved

[L-05] BaseWallet#_validateSignature - Guardian signature mode is encoded and decoded but always rejected by the wallet

Description

The signature format explicitly includes `SignatureMode.guardians`, and `decodeSignature()` accepts and returns this mode as valid input. However, `BaseWallet._validateSignature()` only handles `owner` and `session` modes, and every other mode is forced to fail. As a result, guardian-mode signatures are part of the wire format but have no working validation path, which can mislead integrators into believing guardian signatures are supported when they are actually unusable.

Recommendation

Either implement a real guardian-signature validation flow in the wallet or remove guardian mode from the accepted signature format so unsupported modes cannot be decoded and treated as valid-looking input.

Status

Resolved

[L-06] EntryPoint#innerHandleOp - EntryPoint can report failed account execution as successful when returndata is empty

Description

In `EntryPoint.innerHandleOp()`, the mode starts as `opSucceeded` and is only changed to `opReverted` when a failed account call returns non-empty revert data. If the outer account call fails with empty returndata, such as `revert(0,0)` or an out-of-gas condition, the operation is still reported as successful in `UserOperationEvent` and the paymaster receives `postOp(opSucceeded, ...)`. The real impact in this codebase is incorrect execution status reporting and wrong postOp mode classification, which can mislead monitoring or future integrations that rely on these values.

Recommendation

Treat any failed outer account call as a failed operation regardless of whether revert data is returned, and only use revert data for logging the failure reason.

Status

Resolved

[L-07] EntryPoint#handleAggregatedOps - handleAggregatedOps can report the wrong failing operation index across aggregator groups

Description

In `handleAggregatedOps()`, the contract tracks a global operation index but passes the subgroup-local index into `_validateAccountAndPaymasterValidationData()`. As a result, validation failures in later aggregator groups can revert with an incorrect `FailedOp(opIndex, ...)` value. The real impact is limited to diagnostics and bundler operations, where offchain tooling may attribute the failure to the wrong user operation and remove or reprocess the wrong entry.

Recommendation

Use the global operation index consistently when reporting validation failures in aggregated flows so offchain systems can correctly identify the failing user operation.

Status

Resolved

[L-08] TokenPaymaster#withdrawToken - TokenPaymaster withdraws use raw ERC20 transfer while collection uses SafeERC20

Description

`TokenPaymaster` uses `safeTransferFrom` in `_postOp` to take payment from users but `withdrawToken` and `_withdrawToken` call `IERC20.transfer` directly. Tokens that do not return a bool can make withdrawals revert in Solidity 0.8 even though funds were collected safely, and tokens that return `false` on failure can leave the owner thinking a withdrawal succeeded because `TokenWithdraw` is emitted without checking the transfer result.

Recommendation

Use `safeTransfer` from the existing `using SafeERC20 for IERC20` setup in both withdrawal paths so behavior matches `_postOp` and failed transfers revert without emitting a misleading event.

Status

Resolved

[L-09] DexContract#settleToken - Provider can choose arbitrary low-fee promotional code at settlement

Description

The settlement fee is selected using a `_payinSourceCode` value provided by the provider at settlement time, but this value is never stored or fixed when the order is created. Because of this, the provider can choose any existing low-fee code when settling an order.

Vulnerability Details

In `settleToken()`, the contract reads the fee rate from `payinSourceCodeFeeMap[_payinSourceCode]`, and if no value is found it falls back to the global `feeRate`. The problem is that `_payinSourceCode` is passed directly by the provider during settlement and is not bound to the order in storage. The `Order` struct does not store any payin source code, and neither `swapToken()` nor `swapTokenForUSDT()` records one at creation time. This means the provider can settle an order using any configured code that gives a cheaper fee than intended.

Impact

A provider can repeatedly use the lowest configured promotional fee code for orders that were not meant to receive that discount, causing the protocol to collect less fee while the provider keeps the difference. This does not directly steal user funds, but it can create meaningful ongoing financial loss at scale.

Recommendation

Bind the applicable fee rate or payin source code to the order when the order is created, and use that stored value during settlement instead of accepting a free-form `_payinSourceCode` from the provider.

Status

Resolved

[L-10] RemittanceBase - Upgradeable base contract is missing a storage gap

Description

`RemittanceBase` is used as an upgradeable base contract for proxy-based contracts, but it does not reserve storage slots with a `__gap` array. Because of this, if a future upgrade adds new state variables to the base contract, the storage layout of child contracts can shift and existing proxy storage can become corrupted. This is not an immediate exploit in the current version, but it creates upgrade risk for future versions.

Recommendation

Add a properly sized `__gap` storage array to `RemittanceBase` and preserve storage layout rules for all future upgrades

Status

Resolved

[L-11] Remittance#refundSettle - Refund path ignores `ransom()` return value

Description

In `refundSettle()`, the contract calls `IRemitPair(pairAddr).ransom(...)` but does not check the returned `amountOut` before continuing execution. If the external pair implementation ever returns an insufficient amount without reverting, the contract has no validation that the expected asset was actually recovered, which makes the refund flow rely on unchecked external behavior.

Recommendation

Check the return value of `ransom()` and verify that the recovered amount is sufficient before continuing with the rest of the refund logic.

Status

Resolved

[L-12] Remittance - Implementation contract can be initialized because `_disableInitializers()` is missing

Description

The `Remittance` implementation and its base contract do not disable initializers in a constructor. Because of this, the implementation contract itself can still be initialized directly by anyone, allowing an attacker to become the owner and manager of the implementation's own storage. In this setup the live proxy is initialized separately and keeps isolated storage, so this does not directly compromise proxy user funds, but it remains an upgrade-safety and best-practice issue.

Recommendation

Add a constructor that calls `_disableInitializers()` in the implementation contract hierarchy so the implementation cannot be initialized directly after deployment.

Status

Resolved

[L-13] RemitRateOracle#updateRateByAddress - Unbounded oracle rates allow managers to set drain-pricing for swaps

Description

`RemitRateOracle.updateRateByAddress()` allows a privileged manager to set arbitrary exchange rates with no sanity bounds, enabling swap pricing that can drain a pair's reserves if the controller permits the required trade size.

Vulnerability Details

The oracle stores `rate0` and `rate1` exactly as provided by `onlyManager` callers without validating minimum/maximum values. The pair enforces swap outputs using the oracle-derived formula (effectively `amountOut = amountIn * rate / 1e9` with decimal adjustment), so a manager can set an extreme rate `(e.g., 1000:1)` such that a small input corresponds to an output equal to the entire reserve.

Impact

If an oracle manager key is compromised or a manager behaves maliciously, they can update rates to unfair values and execute (or enable) swaps that extract most or all liquidity from affected pairs in a single transaction, subject to any controller threshold/whitelist constraints.

Recommendation

Add rate sanity checks (min/max bounds and reciprocity/invariant constraints), introduce a delay/timelock for rate changes, and strictly limit/monitor the manager set to reduce the blast radius of a compromised rate setter.

Status

Resolved

[L-14] RemitPair#ransom - `RemitPair.ransom` leaves reserve accounting stale after forced withdrawal

Description

After transferring tokens out of the pair, `ransom` never calls `_update()` to synchronize `reserve0` and `reserve1` with the actual post-withdrawal balances.

`ransom()` has a comment `// force reserves to match balances.` before the emit, suggesting the call was intended but not added.

Vulnerability Details

```
function ransom(address token, uint256 amount, address to) external lock returns
(uint amountOut) {
    ...
    if(amountOut > 0){
        _safeTransfer(token, to, amountOut);
    }
    // force reserves to match balances.
    emit Ransom(msg.sender, token, amountOut, to);
}
```

After `ransom`, the stored reserves still point to the old balances. The mismatch is easy to reproduce in `Remittance.refundSettle`: first the contract calls `ransom` to pull `AEDB` out of the pair, then it transfers `USDT` back into that same pair. Both token balances change, but `reserve0` and `reserve1` are not updated. Until someone calls `sync()`, `getReserves()` returns stale values, and `swap` starts from those stale reserves when running its initial liquidity checks.

Impact

Reserve accounting is incorrect after any ransom call. The `refundSettle` flow leaves the pair in a state where both token balances diverge from their stored reserves simultaneously, and there is no guaranteed sync point before the next swap attempt.

Recommendation

Add a `_update` call at the end of `ransom` after the transfer:

```
if(amountOut > 0){
    _safeTransfer(token, to, amountOut);
}
_update(IERC20(token0).balanceOf(address(this)),
IERC20(token1).balanceOf(address(this)), reserve0, reserve1);
emit Ransom(msg.sender, token, amountOut, to);
```

Status

Resolved

[L-15] RemitRateOracle#batchUpdateRate - `batchUpdateRate` checks only two of the three required array-length equalities

Description

Two length checks are present, but the equality between `providerArr` and `targetCurrencyArr` is never validated. The function assumes all arrays are aligned and indexes all four arrays inside one loop, so mismatched lengths can either revert mid-loop or process only part of the intended batch, depending on which array is shorter.

Vulnerability Details

```
function batchUpdateRate(address[] memory providerArr, string[] memory
baseCurrencyArr,
    string[] memory targetCurrencyArr, uint256[] memory rateArr) onlyManager external
{
    require(providerArr.length == baseCurrencyArr.length, "invalid providerArr or
baseCurrencyArr");
    require(targetCurrencyArr.length == rateArr.length, "invalid targetCurrencyArr or
rateArr");
    // providerArr.length == targetCurrencyArr.length is never checked
    for(uint256 i = 0; i < baseCurrencyArr.length; i++){
        _updateRate(providerArr[i], baseCurrencyArr[i], targetCurrencyArr[i],
rateArr[i]);
    }
}
```

If `providerArr.length == baseCurrencyArr.length` and `targetCurrencyArr.length == rateArr.length` both hold, the function still does not guarantee cross-array alignment. For example, when `baseCurrencyArr.length` is greater than `targetCurrencyArr.length`, the loop eventually reads out of bounds and reverts. When `baseCurrencyArr.length` is smaller than `targetCurrencyArr.length`, extra entries in `targetCurrencyArr` and `rateArr` are ignored because the loop stops at `baseCurrencyArr.length`.

Impact

A manager submitting mismatched arrays can get inconsistent behavior: some malformed inputs revert due to out-of-bounds reads, while others complete but ignore trailing entries.

Recommendation

Add `require(providerArr.length == targetCurrencyArr.length, ...)` alongside the existing checks, or consolidate into a single check: `require(providerArr.length == baseCurrencyArr.length && baseCurrencyArr.length == targetCurrencyArr.length && targetCurrencyArr.length == rateArr.length, ...)`.

Status

Resolved

[L-16] Remittance#withdraw - `Remittance.withdraw` ignores the _amount parameter for native ETH

Description

The `ERC20` path in `withdraw` respects `_amount` and caps it at the contract balance. The `ETH` path ignores `_amount` entirely and drains the entire native balance in one call.

Vulnerability Details

```
function withdraw(address _token, address _recipient, uint256 _amount) external
nonReentrant onlyOwner {
    if(_token == address(0)){
        payable(_recipient).transfer(address(this).balance);  // _amount ignored
    } else {
        uint balance = IERC20(_token).balanceOf(address(this));
        if(_amount > balance) {
            _amount = balance;
        }
        if(_amount > 0) {
            IERC20(_token).safeTransfer(_recipient, _amount);} }
}
```

An owner intending to withdraw a partial `ETH` amount passes `_amount` but the contract ignores it and sends everything.

Impact

Owner-initiated partial `ETH` withdrawals are impossible. Any call with `_token = address(0)` transfers the entire balance regardless of the specified amount, which can drain `ETH` held as operational reserve in a single transaction.

Recommendation

Apply the same capping logic to the `ETH` branch: read `address(this).balance`, cap `_amount` to it, and transfer only `_amount`.

Status

Resolved

[L-17] FiatTokenV1_1#updateDecimals - `updateDecimals` does not rescale the stored `routerFee`

Description

`setRouterFee` scales the fee to the token's current decimal precision at the time of the call. `updateDecimals` changes `decimals` but never touches `routerFee`, so the stored value represents a different real amount after the decimal change.

Vulnerability Details

```
// FiatTokenV1.setRouterFee
function setRouterFee(uint256 newRouterFee) public onlyOwner {
    routerFee = newRouterFee.mul(uint256(10)**decimals);
}

// FiatTokenV1_1.updateDecimals
function updateDecimals(uint8 newTokenDecimals) external virtual onlyOwner returns
(bool) {
    decimals = newTokenDecimals;
    return true;
}
```

If `decimals` was 6 when `setRouterFee(1)` was called, `routerFee = 1_000_000`. After `updateDecimals(8)`, `routerFee` is still `1_000_000` but now represents `0.01` tokens rather than `1` token. Every subsequent `transferWithAuthorization` deducts the wrong fee amount.

Impact

After a decimal change, all fee-bearing authorization transfers collect the wrong fee. If decimals increase the fee becomes smaller than intended; if decimals decrease it becomes larger. The discrepancy is silent and permanent until `setRouterFee` is called again.

Recommendation

In `updateDecimals`, rescale `routerFee` if it is non-zero: `routerFee = routerFee / 10**oldDecimals * 10**newTokenDecimals`. Alternatively, document that `setRouterFee` must be called after any decimal change.

Status

Resolved

[L-18] DexContract#abnormalTokenSettle - `abnormalTokenSettle` lets a provider permanently block the refund path with a wrong signature

Description

`abnormalTokenSettle` takes a raw `_messageHash` from the caller and only checks that the caller signed that value. Since the provider chooses the hash, they can always pass this check by signing any value they want. Later, `_refundPreCheck` verifies the stored signature against a different hash that is computed on-chain from the refund caller address, provider address, and order `ID`. If the stored signature was not made for that exact hash, `_refundPreCheck` returns false and the refund cannot be executed.

Vulnerability Details

In `abnormalTokenSettle`, the signature check proves only that the provider signed the hash they themselves supplied:

```
bytes32 prefixedMessageHash = keccak256(abi.encodePacked("\\\x19Ethereum Signed Message:\\\n32", _messageHash));
address recoveredSigner = ecrecover(prefixedMessageHash, v, r, s);
require(recoveredSigner == msg.sender, "invalid signer");
orders[_orderNo].status = uint256(Status.SETTLE_FAIL);
orders[_orderNo].refundSign = _refundSign;
```

In `_refundPreCheck`, the stored signature is then re-verified against a hash constructed independently on-chain:

```
bytes32 messageHash = keccak256(abi.encodePacked(_refundAddress, orders[_orderNo].provider, bytes(_orderNo)));
...
if(recoveredSigner != orders[_orderNo].provider) return (false, "sign valid failed");
```

Recommendation

Remove `_messageHash` as a parameter and compute it on-chain inside `abnormalTokenSettle` from the same inputs that `_refundPreCheck` expects, so the stored signature is guaranteed to be valid for the downstream check.

Status

Resolved

[L-19] RemitController#setSwapFeeByPair - `setSwapFeeByPair` writes both pair-level and global fee state

Description

`setSwapFeeByPair` updates `swapFee[_pair][_token]`, but it also updates `defaultSwapFee[_token]` in the same call. This coupling is easy to miss and can create configuration confusion, since a pair-specific action is also changing global token fee state.

Vulnerability Details

```
function setSwapFeeByPair(address _pair, address _token, Mode _mode, uint256
_feeValue) onlyManager external {
    //require(_pair != address(0x0), "INVALID_PAIR_ADDRESS");
    require(_token != address(0x0), "INVALID_TOKEN");
    require(_mode == Mode.PERCENT_MODE || _mode == Mode.SIGNLE_MODE, "INVALID_MODE");
    defaultSwapFee[_token] = SwapFee(true, uint256(_mode), _feeValue);  // 默认的也需要
保留一份（会存在冲突）
    swapFee[_pair][_token] = SwapFee(true, uint256(_mode), _feeValue);
    emit UpdateSwapFee(msg.sender, _pair, uint256(_mode), _feeValue);
}
```

Recommendation

If pair-specific and global fee changes are meant to be independent, remove the `defaultSwapFee[_token]` write from `setSwapFeeByPair`. If the current behavior is intentional, document it clearly so operators do not treat `setSwapFeeByPair` as pair-only.

Status

Resolved

[L-20] BaseWallet#setup - BaseWallet with `_lockPeriod = 0` makes `lock()` ineffective

Description

`setup` only calls `_setLockPeriod` when `_lockPeriod > 0`. If zero is passed, `lockPeriod` remains at its default zero value. There is no setter to change it afterward.

Vulnerability Details

```
// BaseWallet.setup
if (_lockPeriod > 0) _setLockPeriod(_lockPeriod);
```

```
// SecurityManager.lock
function lock() public virtual onlyUnlocked {
    _setLock(block.timestamp + lockPeriod, bytes4(keccak256("lock()")));
    emit Locked(address(this), uint64(block.timestamp + lockPeriod));}
// SecurityManager._isLocked
function _isLocked() internal view returns (bool) {
    return lockInfo.release > uint64(block.timestamp);
}
```

With `lockPeriod = 0`, `lock()` sets `release = block.timestamp`. The `_isLocked` check becomes `block.timestamp > block.timestamp`, which is always false. The wallet emits `Locked` and passes the `onlyUnlocked` modifier, but any subsequent call that checks `_isLocked()` or uses `onlyLocked` treats the wallet as unlocked. The lock mechanism is silently inoperative for the lifetime of the wallet.

Recommendation

Add `require(_lockPeriod > 0, "wallet: lockPeriod must be non-zero")` in `setup`, or expose an owner-restricted setter for `lockPeriod` so wallets deployed with zero can correct it without redeployment.

Status

Resolved

[L-21] FiatTokenV1#transferWithAuthorization - Admin can change router fee for previously signed transfers

Description

The `transferWithAuthorization` flow does not include `routerFee` in the signed message. Because `routerFee` is read from storage at execution time, the contract owner can change the fee after a user signs but before the relayer submits the transaction.

Vulnerability Details

In `FiatTokenV1`, the router fee is mutable through `setRouterFee(uint256 newRouterFee)`, which is restricted by `onlyOwner`. When `transferWithAuthorization()` is called, it forwards the current live `routerFee` into `_transferWithAuthorization()`. Inside `_transferWithAuthorization()`, the signature is verified only against `from`, `to`, `value`, `validAfter`, `validBefore`, and `nonce`, but not against `routerFee`. After signature verification, the function transfers `routerFee` to `msg.sender` and only sends `value - routerFee` to the recipient. This means a signed transfer can execute later under a different fee than the one the user may have expected at signing time.

```
function setRouterFee(uint256 newRouterFee) public onlyOwner {
routerFee = newRouterFee.mul(uint256(10)**decimals);
}

function transferWithAuthorization(...) external {
_transferWithAuthorization(..., routerFee);
}

function _transferWithAuthorization(...) internal {
bytes memory data = abi.encode(
TRANSFER_WITH_AUTHORIZATION_TYPEHASH,
from,
to,
value,
validAfter,
validBefore,
```

```
nonce
);
require(SignatureLib.verifySignature(from, DOMAIN_SEPARATOR, v, r, s, data),
"EIP3009: invalid signature");
if (routerFee > 0) _transfer(from, msg.sender, routerFee);
_transfer(from, to, value.sub(routerFee));
}
```

Impact

Previously signed transfers can be executed with a different fee split than the signer may have assumed, which changes how much the relayer receives and how much the recipient gets. In the worst case, increasing the fee can reduce the recipient's net amount or cause pending authorizations to revert if the fee becomes too large relative to `value`.

Recommendation

Include the fee, or the recipient's net amount, in the signed payload so the economic terms cannot change after signing. If the protocol intentionally treats the owner as fully trusted, this should be classified as an informational design note rather than a medium-severity vulnerability.

Status

Acknowledged

[L-22] FiatTokenV1#_transferWithAuthorization - Relayer Fee Can Be Front-Run and Claimed by Any Mempool Observer

Description

The `transferWithAuthorization` and `transferWithAuthorizationBatch` logic pays the `routerFee` to `msg.sender`, but the signed authorization does not bind a specific relayer. Because of this, anyone who sees the transaction in the public mempool can copy it, submit it with a higher gas price, and take the relayer fee.

Vulnerability Details

In `_transferWithAuthorization`, the fee is sent directly to the caller through `if (routerFee > 0) _transfer(from, msg.sender, routerFee);` and the remaining amount is sent to the recipient through `_transfer(from, to, value.sub(routerFee));`. The signed payload only includes `from`, `to`, `value`, `validAfter`, `validBefore`, and `nonce`, so it does not restrict who can submit the authorization. As a result, if an honest relayer broadcasts a valid transaction, an MEV bot can copy the same calldata, submit it with higher gas, become `msg.sender`, and receive the `routerFee`. The honest relayer's transaction will then revert because the nonce has already been used.

Impact

This breaks the relayer incentive model for public mempool execution. Honest relayers can lose their reward and still pay gas for reverted transactions, which makes the meta-transaction flow unreliable and unattractive for third-party relayers.

Recommendation

No action required. The routerFee is intentionally awarded to whichever address submits the transaction first as a competitive incentive, consistent with the protocol's design

Status

Acknowledged

Centralisation

The TOPOS Deal project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the TOPOS Deal project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au

